

SSL/TLS Protocols

Now that we have all the building blocks—symmetric encryption, public keys, hashing, digital signatures, and certificate authorities—we can look at the system that ties them all together: **SSL/TLS**.

TLS (Transport Layer Security) is the cryptographic protocol that secures data moving across the internet. It is the engine behind `https://`. When you log into an account, buy something online, or send an email, TLS is what scrambles that traffic to prevent hackers from snooping on your network.

A Quick Name Clarification: SSL (Secure Sockets Layer) was the original protocol invented by Netscape in the 1990s. It had critical security flaws and was officially deprecated years ago. **TLS** is its modern, highly secure successor. However, because everyone was used to the name "SSL," the tech industry still combines them as **SSL/TLS**, even though your browser uses TLS today.

The Handshake: How a Secure Connection is Born

Before your browser sends a single byte of data to a website (like your password or credit card), they have to agree on how to encrypt it. They do this through a process called the **TLS Handshake**.

Think of it like two spies meeting in a cafe to agree on a secret code before sharing secrets. In modern TLS (specifically TLS 1.3), this incredibly complex process happens in a fraction of a second:

```
Browser — [1. "Hello! Here are my cryptographic capabilities"] —> Server
Browser <— [2. "Hello! Here is my Certificate & Public Key"] — Server
          (Browser verifies the Certificate with a CA)
Browser — [3. Calculates & starts sending encrypted data] ———> Server
```

1. The Client Hello

Your browser connects to the server and says, *"Hey, I want to establish a secure connection. Here is a list of the TLS versions and encryption algorithms (Cipher Suites) that I support."*

2. The Server Hello & Key Exchange

The server responds, picks the strongest mutually supported algorithm, and sends back its **Digital Certificate** (verified by a Certificate Authority) along with its **Public Key**.

3. Authentication & The Hybrid Master Key

Your browser checks the certificate against its built-in list of trusted CAs to make sure it's not a fake site. Once verified, the browser and server use asymmetric math to generate a shared, temporary **Symmetric Session Key**.

4. Secure Transmission

The handshake is over. Both sides switch entirely to symmetric encryption using that brand-new session key. Every message, image, and form you submit from that moment until you close the tab is encrypted with that key.

Why Do We Need the Handshake? (The Hybrid Approach)

As you might recall from our earlier chats, asymmetric encryption (using public/private key pairs) requires heavy mathematical lifting and can slow down a computer if used continuously. On the flip side, symmetric encryption is lightning fast but suffers from the problem of how to share the key safely.

TLS is brilliant because it solves both problems by combining them:

1. It uses **Asymmetric Encryption** for just a split second during the handshake to authenticate the server and safely create a shared key.
2. It immediately shifts to **Symmetric Encryption** to transfer the actual website data at blazing-fast speeds.

When you close your browser tab, that temporary session key is destroyed forever. If you revisit the exact same website two seconds later, a brand-new handshake occurs, and a completely fresh key is created. This ensures that even if a hacker somehow stole a key from yesterday, they couldn't use it to decrypt your data today.